

Inventory Sentinel

Technical Architecture & AI Pipeline Report

Version: 1.0.0

Date: September 2026

Classification: Internal Technical Documentation

Author: Project Engineering Team

Repository: github.com/Suvradeep009/Inventory-Sentinel

Production URL: inventory-sentinel.vercel.app

Table of Contents

1. [Executive Summary](#)
2. [System Architecture & Data Flow](#)
 - 2.1 [Architectural Thesis: Decoupling Deterministic Logic from Stochastic LLMs](#)
 - 2.2 [End-to-End Architectural Pipeline](#)
3. [UI/UX Design Philosophy](#)
4. [Core System Modules – The Four Routes](#)
 - 4.1 [Data Synchronization Hub \(Home\)](#)
 - 4.2 [NLP Sentiment Engine](#)
 - 4.3 [Automated Demand Alerting](#)
 - 4.4 [Insights RAG Terminal](#)
5. [The Tech Stack](#)
6. [Data Architecture & Schema](#)
7. [AI Integration Strategy](#)
8. [Deployment Architecture](#)
9. [API Reference Summary](#)
10. [Appendices](#)

1. Executive Summary

Inventory Sentinel is an AI-powered autonomous inventory dispatch platform that merges traditional relational state management (SQLite-backed CRUD) with a real-time Large Language Model (LLM) sentiment analysis pipeline. The system addresses a critical gap in conventional inventory management software: the inability to incorporate qualitative customer feedback into quantitative procurement decisions.

The platform ingests hardware product catalogs and customer review corpora from vendor APIs, classifies each review into a deterministic 5-tier sentiment taxonomy using Google Gemini's structured JSON output

mode, and computes automated restock signals based on mathematically defined thresholds. A secondary Retrieval-Augmented Generation (RAG) pipeline enables natural-language executive queries against the review corpus, producing grounded conversational summaries without hallucination.

Key Innovation

The core differentiator is the **closed-loop sentiment-to-signal pipeline**: raw unstructured customer text is transformed through an LLM classification layer into three discrete, actionable inventory signals — `HALT_RESTOCK`, `INCREASE_ORDER`, or `HOLD_STEADY` — which directly inform procurement workflows. This eliminates the traditional lag between post-purchase qualitative feedback and upstream supply chain response.

Capabilities at a Glance

Capability	Implementation
Product catalog management	SQLite CRUD with CSV/JSON bulk upload
Vendor data synchronization	REST API ingestion with live stock reconciliation
NLP sentiment classification	Gemini Flash structured JSON – 5-tier taxonomy
Automated demand signaling	Threshold-based conversion (qualitative → quantitative)
Conversational RAG insights	Metadata-filtered retrieval + LLM synthesis
Fallback resilience	Keyword-heuristic classifier when API is unavailable

2. System Architecture & Data Flow

2.1 Architectural Thesis: Decoupling Deterministic Logic from Stochastic LLMs

In enterprise supply chain orchestration, inventory decisions carry direct financial and operational ramifications. Over-ordering locks working capital into depreciating inventory and increases holding costs; under-ordering creates stockouts, supply chain friction, and customer SLA breaches.

A central vulnerability in contemporary generative AI systems is the propensity to entrust Large Language Models (LLMs) with direct operational governance via unconstrained text generation or autonomous tool dispatch. LLMs are fundamentally **stochastic next-token probability engines**. Even when conditioned with near-zero sampling temperatures ($T \approx 0.1$), generative models remain inherently susceptible to:

- Semantic Drift:** Non-deterministic variance in response nuance and classification boundaries across identical review texts.
- Context Hallucination:** Inadvertently inventing product defects or missing subtle negative qualifiers within lengthy user reviews.
- Threshold Inconsistency:** Fluctuation in the severity assigned to identical complaints depending on token positioning and batch context.

CRITICAL ARCHITECTURAL BOUNDARY: STOCHASTIC vs DETERMINISTIC	
NONDETERMINISTIC LLM LAYER (Gemini)	DETERMINISTIC RULE ENGINE (Python)
• Role: Feature Extraction & Text Quantization • Input: Unstructured, noisy customer feedback • Output: Constrained JSON schema [{tier, conf}] • Scope: Zero execution/procurement privilege	• Role: State Evaluation & Signal Action • Input: Discrete 5-tier classification • Logic: Pure mathematical inequalities • Scope: 100% idempotent & auditable

To eliminate these failure modes, **Inventory Sentinel strictly bifurcates the system into two isolated operational tiers:**

- **The Non-Deterministic Feature Extraction Tier:** Google Gemini 1.5 Flash is strictly confined to **linguistic feature extraction**. The LLM is never permitted to determine what inventory action to execute. Instead, it is constrained via schema-enforced structured JSON generation to map natural language customer feedback into a closed, discrete 5-tier classification bucket (`VERY_BAD` , `BAD` , `NEUTRAL` , `GOOD` , `VERY_GOOD`).
- **The Deterministic Inventory Decision Tier:** A pure, compiled mathematical rule engine operates exclusively on the resulting discrete distribution arrays. Restock signals (`HALT_RESTOCK` , `INCREASE_ORDER` , `MAINTAIN_EQUILIBRIUM`) are derived strictly via mathematically invariant inequality evaluations. This guarantees that every operational decision is **100% repeatable, fully auditable, and mathematically verifiable**.

2.2 End-to-End Architectural Pipeline

The system is structured as an asynchronous, five-stage reactive pipeline connecting external vendor data feeds to administrative decision interfaces:



Detailed Stage Breakdown

1. Presentation & Client State Layer

The user interface is engineered with **React 19** and **Vite / Next.js Serverless** architecture, executing within the browser under the Editorial Neo-Brutalist design framework.

- **State Hydration:** Client components maintain synchronized state trees across product lists, review buckets, sentiment metrics, and operational dispatch logs.

- **Local Storage Caching:** The frontend implements `localStorage` caching to persist operator configurations, custom vendor API endpoint overrides (e.g., swapping between `dummyjson.com/products` and local simulated feeds), and active Gemini API keys without requiring persistent database mutations.
- **Data-Dense Tabular Rendering:** Utilizes componentized tables (`ReviewTable`, `SignalCard`, `TerminalInput`) rendering tabular catalogs with sub-millisecond client filtering, sorting, and responsive pagination.

2. Data Ingestion Subsystem

The data ingestion engine interfaces with external vendor repositories and public catalogs (e.g., `dummyjson.com/products`) or proprietary vendor REST feeds).

- **Asynchronous Extraction:** REST clients execute concurrent asynchronous HTTP GET operations against remote catalog endpoints.
- **Schema Standardization:** Ingested JSON structures are mapped into normalized internal domain entities: $\text{\texttt{\$}\{Payload\} \mapsto \left\{ \text{\texttt{id}}, \text{\texttt{sku}}, \text{\texttt{title}}, \text{\texttt{category}}, \text{\texttt{price}}, \text{\texttt{stock}}, \text{\texttt{reviews}}: \left[\left\{ \text{\texttt{reviewer}}, \text{\texttt{text}}, \text{\texttt{rating}}, \text{\texttt{timestamp}} \right\} \right] \right\} \text{\texttt{\$}}$
- **Transactional Persistence:** Stored within an embedded SQLite database with automated transaction rollbacks upon malformed records, ensuring catalog integrity across ingest cycles.

3. NLP Sentiment Classification Pipeline

Raw customer reviews are normalized, deduplicated, and dispatched through a batched classification pipeline:

- **Batch Ingestion:** Reviews are chunked into batches of 10 items to balance Gemini 1.5 Flash throughput, eliminate per-request latency overhead, and operate well within token budget constraints.
- **Strict JSON Response Schema Enforcement:** Leveraging Gemini's `response_mime_type: "application/json"`, the LLM is constrained to emit an exact JSON array of typed objects:

```
[
  {
    "tier": "VERY_BAD" | "BAD" | "NEUTRAL" | "GOOD" | "VERY_GOOD",
    "confidence": 0.94
  }
]
```

- **Quantization:** This converts noisy, colloquial text into discrete, low-cardinality mathematical categorical variables. Any response failing strict Pydantic validation triggers an immediate fallback to the local deterministic keyword engine.

4. Deterministic Inventory Decision Engine

A decoupled mathematical module evaluates aggregate catalog sentiment against rigid risk-adjusted procurement boundaries:

- **Distribution Computation:** Given a product with N classified reviews and class counts C_{τ} for $\tau \in \{\text{VB}, \text{B}, \text{NE}, \text{G}, \text{VG}\}$: $R_{\text{neg}} = \frac{C_{\text{VERY_BAD}} + C_{\text{BAD}}}{N}$, $R_{\text{pos}} = \frac{C_{\text{GOOD}} + C_{\text{VERY_GOOD}}}{N}$
- **Evaluation Inequalities:** $\text{Inventory Signal} = \begin{cases} \text{HALT_RESTOCK} & \text{if } R_{\text{neg}} \geq 0.45 \\ \text{INCREASE_ORDER} & \text{if } \end{cases}$

$R_{\{\text{pos}\}} \geq 0.65 \quad (\text{Color: Mint Green} \#88E6B8) \quad \mathbf{MAINTAIN_EQUILIBRIUM} \& \text{otherwise} \quad (\text{Color: Ink Black} \#111111) \end{cases} \$\$$

- **Priority Override:** The negative threshold is evaluated first. Even if a product has high positive volume, a negative ratio exceeding 45% forces an immediate procurement freeze, preventing supply chain capital loss due to defective equipment.

5. Contextual Retrieval & RAG Pipeline

To empower executive operators with conversational inquiries into product sentiment without risking cross-product contamination:

- **Vectorization & Storage:** Review texts are chunked and linked with explicit metadata vectors containing `product_id`, `product_name`, `sku`, `reviewer`, `tier`, and `timestamp`.
 - **Metadata Pre-Filtering:** Before computing semantic distances, a strict metadata filter isolates the chunk universe: $\Omega_{\{\text{retrieval}\}} = \{c \in \text{Corpus} \mid c.\text{product_id} = \text{Target_ID}\} \$\$$. This pre-filter eliminates 100% of cross-product context leakage, ensuring that complaints regarding an Inverter cannot bleed into queries regarding a Battery.
 - **Semantic Cosine Similarity & Intent Boosting:** Chunks within $\Omega_{\{\text{retrieval}\}} \$$ are scored against user query tokens using term overlap and query-intent polarity weighting to identify the top- k ($k=3$) most informative excerpts.
 - **Context Injection & Synthesis:** The top-3 chunks are injected into a constrained synthesis prompt template sent to Gemini 1.5 Flash. The prompt instructs the model to produce a professional, third-person executive briefing while explicitly prohibiting reference to internal chunk identifiers or raw confidence metrics.
-

3. UI/UX Design Philosophy

The Editorial Neo-Brutalist Aesthetic

Inventory Sentinel's frontend deliberately rejects the prevailing trends of gamified dashboards, gradient-heavy glassmorphism, and dark-mode SaaS templates. Instead, it adopts a **print-inspired, grid-locked editorial broadsheet** design language optimized for data legibility and administrative focus.

Design Rationale

Inventory management is an inherently analytical discipline. Operators need to scan high-density tabular data, compare polarity distributions across products, and act on color-coded urgency signals — often under time pressure during procurement cycles. The editorial approach treats each screen as a **newspaper front page**: structured columns, clear typographic hierarchy, and deliberate use of whitespace to prevent cognitive overload.

Design System Tokens

Token	Value	Purpose
--bg-paper	#F7F4EB	Organic cream paper surface — reduces eye strain on data-heavy screens
--ink-black	#111111	High-contrast editorial ink for maximum readability
--mint-accent	#88E6B8	Positive sentiment indicator (Good / Very Good / Increase Order)
--coral-accent	#FF9E9E	Negative sentiment indicator (Bad / Very Bad / Halt Restock)
--shadow-card	3px 3px 0px #111	Neo-brutalist offset shadow — tactile depth without blur
--radius-card	14px	Modern structured SaaS card rounding

Typography Stack

The platform employs a four-tier typographic system to establish visual hierarchy:

Layer	Typeface	Role
Display headings	Syne 700/800	Route titles, page mastheads — geometric, bold, editorial
Serif accents	Playfair Display 700/900	Signal names, emphasis blocks — classical authority
Body text	Plus Jakarta Sans 400–800	Paragraphs, labels, navigation — clean geometric sans-serif
Data / Monospace	JetBrains Mono 400–700	SKU codes, API status, terminal output, numerical data

Component Architecture

The frontend is built with **React 19** and **Vite 8**, using a component-based architecture:

Component	File	Purpose
Navbar	Navbar.jsx	Sticky masthead with 4 numbered navigation links and live Gemini status pill
SignalCard	SignalCard.jsx	Demand signal display card with color-coded action buttons
SentimentBadge	SentimentBadge.jsx	Tier badge component (VERY_BAD through VERY_GOOD)
ReviewTable	ReviewTable.jsx	Tabular review display with sentiment classifications
TerminalInput	TerminalInput.jsx	Monospace terminal-style input for RAG queries
EmptyState	EmptyState.jsx	Placeholder component for empty data views

4. Core System Modules – The Four Routes

4.1 Data Synchronization Hub (Home)

Route: `/` (Home Hub)

Backend Endpoints: `GET /api/vendor-data`, `POST /api/vendor-data/apply`, `POST /api/analyze`

Component: `HomePage.jsx` (759 lines)

Purpose

The Home Hub serves as the operational command center, implementing a **three-phase data synchronization pipeline** that fetches, classifies, and commits vendor catalog data in a single automated workflow.

Synchronization Pipeline

The `LiveStoreSyncModule` component orchestrates the following pipeline:

Phase 1 — FETCHING: The system requests raw catalog payload from a configurable vendor endpoint. The default internal endpoint (`/api/vendor-data`) provides a simulated "Apex Power & Grid External Vendor Feed (v2.4)" containing 5 hardware products (Inverter, Battery, Generator, Charger, Power Supply) and 35 domain-specific customer reviews. Operators can override this URL to point at an external vendor REST API.

Phase 2 — ANALYZING: All 35 review texts are extracted and dispatched to the Gemini NLP classification engine via `POST /api/analyze`. The engine processes reviews in batches of 10 using structured JSON output, returning a 5-tier sentiment classification with confidence scores for each review.

Phase 3 — COMMITTING: Classified data is committed to the SQLite database via `POST /api/vendor-data/apply`. The endpoint performs an upsert: matching products by SKU are updated with fresh stock counts; new products are inserted. Reviews are appended to the review table. The in-memory sentiment cache is fully invalidated to force recalculation of demand signals.

Additional Capabilities

- **Manual Product Entry:** A form for adding individual stock items with auto-generated SKU codes
- **Bulk CSV/JSON Upload:** `POST /api/upload/products` and `POST /api/upload/reviews` for file-based data ingestion
- **System Diagnostics Dashboard:** Real-time display of total products, reviews, and signal distribution across the catalog

4.2 NLP Sentiment Engine

Route: `/sentiment`

Backend Module: `nlp_engine.py` (398 lines)

Backend Endpoint: `GET /api/products/{id}/sentiment`

Component: `SentimentPage.jsx` (401 lines)

Prompt Engineering Strategy

The NLP engine employs a **strict structured output** strategy, leveraging Gemini Flash's `response_mime_type: "application/json"` parameter to force the LLM into deterministic, parseable classifications rather than free-form text.

System Prompt (Verbatim)

```
You are a strict sentiment classifier for customer product reviews.
Classify each review into EXACTLY one of these five tiers:
VERY_BAD – severe complaints, product failure, would not recommend
BAD       – clear disappointment, notable issues, negative overall
NEUTRAL   – mixed, average, "gets the job done", no strong opinion
GOOD      – positive experience, satisfied, recommends
VERY_GOOD – exceptional praise, loves product, would strongly recommend

Return a JSON array with one object per review, in the same order as input.
Each object must have exactly these keys:
"tier"      : one of [VERY_BAD, BAD, NEUTRAL, GOOD, VERY_GOOD]
"confidence": float between 0.0 and 1.0

Do NOT include any explanation, markdown, or extra keys. Return ONLY the JSON array.
```

Design Decisions

Decision	Rationale
5-tier taxonomy (not 3 or binary)	Captures gradient severity — distinguishes "slightly disappointed" from "product is a safety hazard"
JSON schema enforcement	<code>response_mime_type: "application/json"</code> eliminates markdown wrapping, prose preambles, and schema drift
Temperature = 0.1	Near-deterministic output — repeated classifications of the same review text yield identical tier assignments
Confidence clamping (<code>min 0.50, max 0.99</code>)	Prevents the model from expressing pathological certainty (1.0) or uncertainty below the noise floor
Batch processing (chunks of 10)	Balances API throughput against prompt context window utilization; prevents token overflow

Dual-Engine Architecture

The NLP engine implements a **graceful degradation** pattern:

1. **Primary Engine:** Google Gemini Flash (structured JSON classification)
2. **Fallback Engine:** Keyword-weighted heuristic classifier

The heuristic fallback uses four curated word banks (153 total keywords across VERY_POSITIVE, POSITIVE, NEGATIVE, and VERY_NEGATIVE categories plus NEUTRAL signals) with weighted scoring:

```
scores = {
  "VERY_GOOD": vp * 3.0 + p * 0.5,
  "GOOD":      p * 2.5 + vp * 0.8,
  "NEUTRAL":   ne * 3.0 + 1.0,      # +1.0 bias toward neutral
  "BAD":       n * 2.5 + vn * 0.8,
  "VERY_BAD":  vn * 3.0 + n * 0.5,
}
```

A deterministic seed derived from the review text's MD5 hash ensures that repeated heuristic classifications of identical text produce identical results — critical for cache coherence.

Sentiment Visualization

The Sentiment Page presents classified data in two view modes:

- **5-Column Buckets:** Visual column layout with reviews sorted into their respective tier bins, color-coded with coral (negative) and mint (positive) accents
- **Magazine Data Table:** Tabular view with per-review tier badges, confidence percentages, and reviewer attribution

Each product displays four summary metrics:

- **Overall Rating** (weighted average on a 1.0–5.0 scale using tier weights: VERY_BAD=1, BAD=2, NEUTRAL=3, GOOD=4, VERY_GOOD=5)
- **Autonomous Signal** (HALT_RESTOCK / INCREASE_ORDER / HOLD_STEADY)
- **Negative Ratio** (percentage of VERY_BAD + BAD reviews, threshold indicator at $\geq 45\%$)
- **Positive Ratio** (percentage of GOOD + VERY_GOOD reviews, threshold indicator at $\geq 65\%$)

4.3 Automated Demand Alerting

Route: `/demand`

Backend Endpoint: `GET /api/inventory`

Component: `DemandPage.jsx` (270 lines)

Signal Computation Algorithm

The `compute_restock_signal()` function in `nlp_engine.py` converts qualitative sentiment distributions into one of three discrete, actionable inventory signals:

```
def compute_restock_signal(distribution: dict, total: int) → str:
    if total == 0:
        return "HOLD_STEADY"

    negative_pct = (distribution["VERY_BAD"] + distribution["BAD"]) / total
    positive_pct = (distribution["GOOD"] + distribution["VERY_GOOD"]) / total

    if negative_pct ≥ 0.45:
        return "HALT_RESTOCK"
    elif positive_pct ≥ 0.65:
        return "INCREASE_ORDER"
    return "HOLD_STEADY"
```

Signal Definitions & Thresholds

Signal	Condition	Color Code	Procurement Action
HALT_RESTOCK	$(\text{VERY_BAD\%} + \text{BAD\%}) \geq 45\%$	 Coral (#FF9E9E)	Freeze all purchase orders; initiate QA audit
INCREASE_ORDER	$(\text{GOOD\%} + \text{VERY_GOOD\%}) \geq 65\%$	 Mint (#88E6B8)	Accelerate procurement volume +35%
HOLD_STEADY	Everything else	 Neutral (#E5E2D9)	Maintain current baseline schedules

Mathematical Properties

- Mutual exclusivity:** The three signals are mutually exclusive by construction — **HALT_RESTOCK** is evaluated first (priority override), preventing a product with 50% negative and 70% positive from receiving a contradictory **INCREASE_ORDER** signal.
- Asymmetric thresholds:** The negative threshold (45%) is deliberately lower than the positive threshold (65%). This reflects a risk-averse procurement philosophy: it takes fewer bad reviews to halt restocking than good reviews to increase orders. Customer complaints carry higher signal weight than praise.
- Zero-review default:** Products with no reviews default to **HOLD_STEADY** rather than **INCREASE_ORDER**, implementing a conservative "do not act without evidence" policy.

Visualization

The Demand Alerts page renders:

- Signal distribution summary:** Count of products in each signal category
- Filterable inventory grid:** **SignalCard** components with color-coded borders and one-click action dispatch buttons
- Recharts bar chart:** Stacked visualization of negative%, positive%, and stock levels across all products

4.4 Insights RAG Terminal

Route: `/insights` (aliased as `/rag`)

Backend Module: `rag_engine.py` (301 lines)

Backend Endpoint: `POST /api/rag`

Component: `RagPage.jsx` (312 lines)

RAG Pipeline Architecture

The Retrieval-Augmented Generation pipeline implements a **metadata pre-filtering** strategy that ensures the LLM only synthesizes answers from reviews belonging to the queried product — preventing cross-product hallucination.

Stage 1 — Product Extraction

```
def extract_target_product(query: str, product_id: int | None = None):
```

The system extracts the target product from the user's natural language query using three matching strategies (in priority order):

1. **Explicit product_id** parameter (if provided by frontend dropdown)
2. **Name substring match** against the product catalog
3. **SKU / Order ID match** against structured identifiers
4. **Numeric pattern match** (e.g., "product #3" → product_id 3)

Stage 2 – Metadata Pre-Filter + Retrieval

```
def simulate_retrieval(query: str, product_id: int | None = None):
```

Once a target product is identified, the retriever applies a **metadata pre-filter** (`where={'product_name': target_product}`) to scope the chunk universe exclusively to that product's reviews. This is functionally equivalent to a vector database's metadata filtering but implemented without an external embedding store.

Each review is chunked with rich metadata:

```
{
  "text":      review["text"],
  "product_name": product["name"],
  "sku":       product["sku"],
  "reviewer":  review["reviewer"],
  "tier":      sentiment["tier"],          # Heuristic pre-classification
  "confidence": sentiment["confidence"],
  "timestamp": review["timestamp"],
}
```

Relevance scoring uses keyword-overlap with stopwords filtering:

```
score = overlap / (len(query_words) + 0.5 * len(chunk_words - query_words) + 1e-9)
```

Polarity boosting adjusts relevance based on detected query intent:

- Queries containing "bad", "fail", "problem" → +0.35 boost for VERY_BAD/BAD chunks
- Queries containing "good", "great", "excellent" → +0.35 boost for VERY_GOOD/GOOD chunks
- Queries containing "halt", "restock" → +0.25 boost for negative chunks
- Queries containing "increase", "order", "surge" → +0.25 boost for positive chunks

The **top 3 chunks** by relevance are selected for synthesis.

Stage 3 – Conversational Executive Synthesis

The system prompt instructs Gemini to produce **natural, professional executive summaries**:

```
You are an inventory analyst. Based on the retrieved review chunks, write a brief, professional executive summary of the customer sentiment. Do not reference chunks, confidence scores, or raw data formatting. Speak naturally.
```

The prompt template constructs a grounded context window:

```
Target Equipment: {product_name} (Order ID: {order_id}, SKU: {sku})
```

```
Customer Feedback Samples:
```

- "{chunk_1_text}" (Product: {product_name})
- "{chunk_2_text}" (Product: {product_name})
- "{chunk_3_text}" (Product: {product_name})

```
Inventory Manager Query:
```

```
{user_query}
```

```
Executive Summary:
```

This ensures the LLM's response is **grounded exclusively in retrieved evidence** rather than parametric knowledge.

Frontend Streaming Simulation

The RAG page simulates character-by-character streaming of the LLM response using a client-side interval that reveals 8 characters every 14ms, creating a typewriter effect that communicates active synthesis to the operator.

5. The Tech Stack

Frontend

Technology	Version	Purpose
React	19.2.8	Component-based UI framework
React Router DOM	7.18.3	Client-side SPA routing (4 routes + alias)
Vite	8.2.2	Build tool with HMR, production bundling
Recharts	3.10.1	Declarative charting (demand visualization)
Oxlint	1.79.0	Rust-based linter for code quality
Google Fonts	—	Syne, Playfair Display, Plus Jakarta Sans, JetBrains Mono

Backend

Technology	Version	Purpose
Python	3.11+	Server runtime
FastAPI	0.115.0	ASGI web framework with auto-generated OpenAPI docs
Uvicorn	0.30.0	Lightning-fast ASGI server
SQLite	Built-in	Embedded relational database (zero-config)
Pydantic	(via FastAPI)	Request/response validation via type-annotated models
python-dotenv	1.0+	Environment variable management
python-multipart	0.0.9	File upload parsing (CSV/JSON ingestion)

AI Integration

Technology	Model	Purpose
Google Generative AI SDK	0.8+	Python client for Gemini API
Gemini Flash	gemini-3.6-flash / gemini-1.5-flash	Sentiment classification (structured JSON) and RAG synthesis
API Key Source	Google AI Studio	Single unified API key for both NLP and RAG pipelines

Deployment

Technology	Purpose
Vercel	Production hosting with serverless Python functions
Vercel CI/CD	Automated builds on <code>git push</code> via GitHub integration
Environment Variables	<code>GEMINI_API_KEY</code> managed via Vercel dashboard
GitHub	Source control and deployment trigger

6. Data Architecture & Schema

SQLite Schema

```
CREATE TABLE products (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  name        TEXT NOT NULL,  
  sku         TEXT NOT NULL UNIQUE,  
  category    TEXT NOT NULL,  
  order_id    TEXT NOT NULL DEFAULT '#7676',  
  price       REAL NOT NULL,  
  stock       INTEGER NOT NULL DEFAULT 0,  
  status      TEXT NOT NULL DEFAULT 'In Stock'  
);  
  
CREATE TABLE reviews (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  product_id  INTEGER NOT NULL,  
  reviewer    TEXT NOT NULL,  
  text        TEXT NOT NULL,  
  timestamp   TEXT NOT NULL,  
  FOREIGN KEY (product_id) REFERENCES products(id)  
);
```

Seed Dataset

The database auto-seeds with a **30-product hardware and power equipment catalog** in Indian Rupees (₹) spanning 5 categories, accompanied by **126 domain-specific customer reviews** from CSV datasets in `backend/data/`.

7. AI Integration Strategy

Unified API Key Architecture

Both the NLP classification engine and the RAG synthesis engine share a single Google Gemini API key, configured via the `GEMINI_API_KEY` environment variable. This simplifies credential management while allowing independent model configuration:

Engine	Temperature	Output Format	Model Priority
NLP Classifier	0.1 (near-deterministic)	<code>application/json</code>	gemini-3.6-flash → gemini-flash-latest → gemini-1.5-flash
RAG Synthesizer	0.2 (slight creativity)	Free-form text	Same cascade

Hot-Reload Capability

The `POST /api/key` endpoint enables runtime API key rotation without server restart. The `nlp_engine.reinitialize(key)` function reinitializes the Gemini client and clears the sentiment cache.

atomically.

In-Memory Caching

Sentiment summaries are cached in a Python dictionary (`_sentiment_cache`) keyed by `product_id` . Cache invalidation is triggered explicitly when:

- A new review is added (`POST /api/reviews`)
- Vendor data is synchronized (`POST /api/vendor-data/apply`)
- Reviews are uploaded via file (`POST /api/upload/reviews`)

8. Deployment Architecture

Vercel Serverless Configuration

```
{
  "version": 2,
  "buildCommand": "cd frontend && npm ci && npm run build",
  "outputDirectory": "frontend/dist",
  "functions": {
    "api/index.py": {
      "maxDuration": 30
    }
  },
  "rewrites": [
    { "source": "/api/:path*", "destination": "/api/index.py" },
    { "source": "/((?!api/).*)", "destination": "/index.html" }
  ]
}
```

Build Pipeline

1. **Frontend build:** `npm ci` (deterministic install) → `npm run build` (Vite production bundle)
2. **Python function:** Vercel auto-detects `api/index.py` , installs `api/requirements.txt` via `uv`
3. **Static assets:** `frontend/dist/` served via Vercel's global CDN
4. **API routing:** `/api/*` requests routed to the Python serverless function
5. **SPA fallback:** All non-API routes serve `index.html` for client-side routing

Environment Variables

Variable	Scope	Description
<code>GEMINI_API_KEY</code>	Production / Preview / Development	Google AI Studio API key for Gemini Flash
<code>INVENTORY_DB_PATH</code>	Vercel only	Overrides SQLite path to <code>/tmp/inventory.db</code> (writable sandbox)

9. API Reference Summary

System Endpoints

Method	Endpoint	Description
GET	/api/status	System statistics, signal distribution, engine status
GET	/api/key-status	Gemini API key availability and engine identifier

Product Endpoints

Method	Endpoint	Description
GET	/api/products	List all products
GET	/api/products/{id}	Get single product
POST	/api/products	Create new product

Review Endpoints

Method	Endpoint	Description
GET	/api/products/{id}/reviews	List reviews for a product
POST	/api/reviews	Add review with instant sentiment classification

NLP / Sentiment Endpoints

Method	Endpoint	Description
GET	/api/products/{id}/sentiment	Full sentiment summary with 5-tier distribution
POST	/api/analyze	Batch classify arbitrary review texts

Demand / Inventory Endpoints

Method	Endpoint	Description
GET	/api/inventory	All products with computed signals and polarity percentages

RAG Endpoints

Method	Endpoint	Description
POST	/api/rag	Execute RAG query with metadata pre-filtering

Vendor / Upload Endpoints

Method	Endpoint	Description
GET	/api/vendor-data	Simulated external vendor feed
POST	/api/vendor-data/apply	Ingest and commit vendor catalog
POST	/api/upload/products	Bulk upload products (CSV/JSON)
POST	/api/upload/reviews	Bulk upload reviews (CSV/JSON)

10. Appendices

A. File Structure

```

inventory-sentinel/
├── api/
│   ├── index.py           # Vercel serverless endpoint
│   └── requirements.txt    # Python deps for Vercel
├── backend/
│   ├── main.py            # FastAPI application (669 lines)
│   ├── nlp_engine.py      # NLP classifier (398 lines)
│   ├── rag_engine.py      # RAG retrieval + synthesis (301 lines)
│   ├── database.py        # SQLite ORM + seed data (516 lines)
│   ├── models.py          # Pydantic schemas (105 lines)
│   ├── requirements.txt    # Python dependencies
│   ├── .env               # API key (gitignored)
│   └── data/
│       ├── products_dataset_30.csv
│       └── reviews_dataset_126.csv
├── frontend/
│   ├── index.html         # SPA entry point with Google Fonts
│   ├── package.json       # React 19, Vite 8, Recharts
│   ├── vite.config.js     # Vite configuration
│   └── src/
│       ├── App.jsx        # Route definitions
│       ├── api.js         # API client (169 lines)
│       ├── index.css      # Design system (1193 lines)
│       ├── pages/
│       │   ├── HomePage.jsx # 759 lines
│       │   ├── SentimentPage.jsx # 401 lines
│       │   ├── DemandPage.jsx # 270 lines
│       │   └── RagPage.jsx    # 312 lines
│       └── components/
│           ├── Navbar.jsx
│           ├── SignalCard.jsx
│           ├── SentimentBadge.jsx
│           ├── ReviewTable.jsx
│           ├── TerminalInput.jsx
│           └── EmptyState.jsx
├── vercel.json            # Deployment configuration
├── run.bat                # Local one-click launcher
└── .gitignore

```

B. Heuristic Keyword Banks

Category	Count	Example Keywords
VERY_POSITIVE	24	amazing, incredible, outstanding, exceptional, perfect, flawless, blown away
POSITIVE	27	great, solid, reliable, efficient, sturdy, responsive, well-made
NEGATIVE	26	disappointing, poor, cheap, flimsy, frustrating, overpriced, broke
VERY_NEGATIVE	21	terrible, horrible, worst, awful, useless, waste, garbage, dead
NEUTRAL	11	decent, okay, average, fine, not bad, acceptable

C. Pydantic Model Inventory

Model	Fields	Used By
<code>ProductCreate</code>	name, category, price, stock, sku?, order_id?, status?	<code>POST /api/products</code>
<code>ReviewCreate</code>	product_id, reviewer, text	<code>POST /api/reviews</code>
<code>BatchAnalyzeRequest</code>	reviews: list[str]	<code>POST /api/analyze</code>
<code>RagRequest</code>	query, product_id?	<code>POST /api/rag</code>
<code>VendorSyncRequest</code>	products, reviews, analyzed?	<code>POST /api/vendor-data/apply</code>

Document generated September 2026. Inventory Sentinel v1.0.0.